Detection of Forged Handwritten Signatures Using a Reference-Based Convolutional Neural Network

Akshara Thapan (Roll No. 7025000032)

UCS321 AI for Engineers, Report 1, Topic T-178

Abstract

This report presents a convolutional neural network (CNN) for detecting forged handwritten signatures in the CEDAR database. Analysis of the raw scans revealed a background artefact that permits perfect classification without reference to the handwriting; this was removed by binarisation. Evaluated on eight signers excluded from training, a CNN that compares each signature with a genuine reference achieved an accuracy of 78.8% and a ROC AUC of 0.861, outperforming a single-image CNN and all classical baselines.

1. Introduction

Handwritten signatures remain a common means of authentication in banking and legal practice, yet manual verification is slow and subjective. The objective of this work is a binary classifier that separates genuine signatures from skilled forgeries. Because a deployed system must handle new customers without retraining, all evaluation was performed on signers absent from training.

2. Dataset

The CEDAR signature database [1] contains 55 signers, each with 24 genuine signatures and 24 skilled forgeries. A public GitHub copy pinned to one commit was used [2]. Six stray files that duplicated other images were removed, leaving 2,640 images. The classes are balanced, so chance accuracy is 50%.

An initial data check found that the median background grey level was near 239 for genuine images and 254 for forged images, with no overlap (Figure 1). A single brightness threshold therefore achieved 100.0% test accuracy without examining any handwriting. This indicates a difference in scanning, not a property of the signatures, and it had to be removed.

3. Methodology

Each image was binarised with Otsu's method [4], cropped to the signature and resized to 96 × 192 pixels with padding, which gives every image an identical background. The IQR rule flagged 102 images on aspect ratio and 61 on ink coverage; these were retained as legitimate variation. The data were split by signer into 39 training, 8 validation and 8 test

signers, so that no individual appears in more than one set.

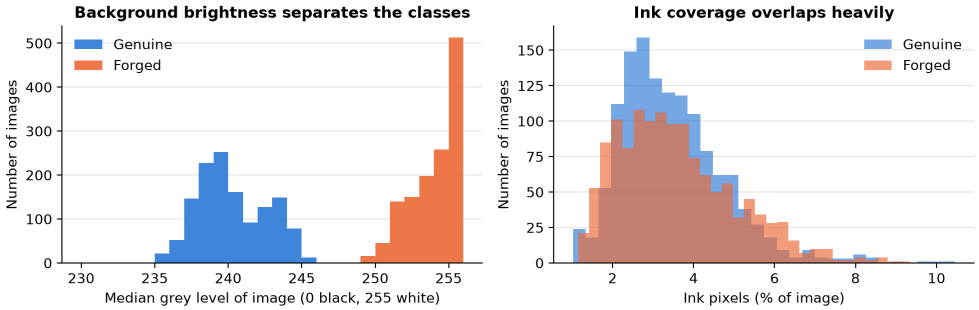


Figure 1: Background grey level (left) separates the classes completely across 2,640 images; ink coverage (right) overlaps heavily.

Three CNNs were built from the same four convolutional blocks. The single-image CNN receives only the questioned signature. The Reference CNN, the principal model, receives the questioned signature and a genuine signature of the same person as a two-channel input, reflecting the comparison made in practice and the writer-independent approach of SigNet [3]; its output is averaged over five enrolled references. A third CNN, trained on raw images, served only as a control. Training used Adam with a cosine learning-rate schedule, dropout and light geometric augmentation, and the best epoch was selected on validation accuracy. Logistic Regression, an RBF SVM, a Random Forest and a Reference SVM were used for comparison [5].

4. Results

Table 1 shows that the Reference CNN was the best valid model, correctly classifying 271 of 344 test signatures (78.8%), slightly below the expected 80%. Access to a reference improved accuracy by 8.7 points over the single-image CNN. The Reference SVM reached only 55.5% with equivalent input, which suggests that the convolutional filters contribute substantially to the comparison.

Table 1: Results on 344 signatures from unseen signers (forged is positive). FAR: forgeries accepted. FRR: genuine rejected. *Invalid: uses the artefact.

Model	Acc.	Prec.	Rec.	F1	AUC	FAR	FRR
Reference CNN	**0.788**	**0.805**	**0.818**	**0.811**	**0.861**	**0.182**	**0.250**
Single-image CNN	0.701	0.694	0.828	0.755	0.792	0.172	0.461
Random Forest	0.581	0.638	0.578	0.607	0.564	0.422	0.414
Reference SVM	0.555	0.577	0.760	0.656	0.606	0.240	0.704
SVM (RBF)	0.546	0.596	0.583	0.590	0.574	0.417	0.500
Logistic Regression	0.474	0.531	0.484	0.507	0.479	0.516	0.539
Raw control*	1.000	1.000	1.000	1.000	1.000	0.000	0.000

Errors were reasonably balanced, with 35 forgeries accepted and 38 genuine signatures rejected (Figure 2). Accuracy ranged from 60.5% to 100.0% across the test signers. Repeating training on three signer splits gave a mean accuracy of 75.7% with a standard deviation of 2.8 points. A random image split raised the single-image CNN from 70.1% to 76.7%, showing that 6.7 points would have been a product of leakage.

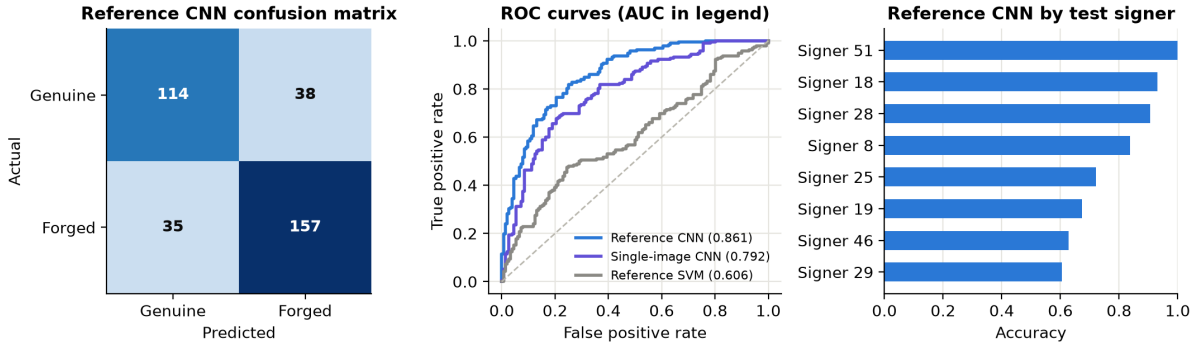


Figure 2: Reference CNN on the test signers: confusion matrix (left), ROC curves with AUC in the legend (centre) and accuracy by signer (right).

5. Discussion and Conclusion

The Reference CNN is the recommended model. The principal finding concerns method: the 100.0% obtained from raw images is not meaningful, and the valid result on unseen signers is 78.8%. The shortfall from 80% is attributed to the small number of training signers, the similarity of skilled forgeries to genuine signatures, and the removal of the artefact. The model was not tuned further against the test set, as this would invalidate the estimate.

The study is limited by its eight test signers and by the possibility that the results do not transfer to other scanners. Binarisation also discards pen-pressure information. Future work should evaluate a pretrained network, use a larger multi-writer dataset, and set the decision threshold according to the cost of an accepted forgery.

References

[1] M. K. Kalera, S. Srihari and A. Xu, “Offline signature verification and identification using distance statistics,” Int. J. Pattern Recognition and Artificial Intelligence, vol. 18, no. 7, pp. 1339–1360, 2004.

[2] CEDAR signature images. Official: https://cedar.buffalo.edu/NIJ/data/signatures.rar. GitHub copy used: https://github.com/Snehal-Aparadh/SignatureForgeryDetection (commit 9e4157b).

[3] S. Dey et al., “SigNet: Convolutional Siamese network for writer independent offline signature verification,” arXiv:1707.02131, 2017.

[4] N. Otsu, “A threshold selection method from gray-level histograms,” IEEE Trans. Systems, Man, and Cybernetics, vol. 9, no. 1, pp. 62–66, 1979.

[5] PyTorch, https://pytorch.org; F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” J. Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

AI tool use: Claude (Anthropic) was used to help design the experiments, write the notebook code and draft this report.